

App Minhas Viagens

Mapas e Geolocalização

Parte 2

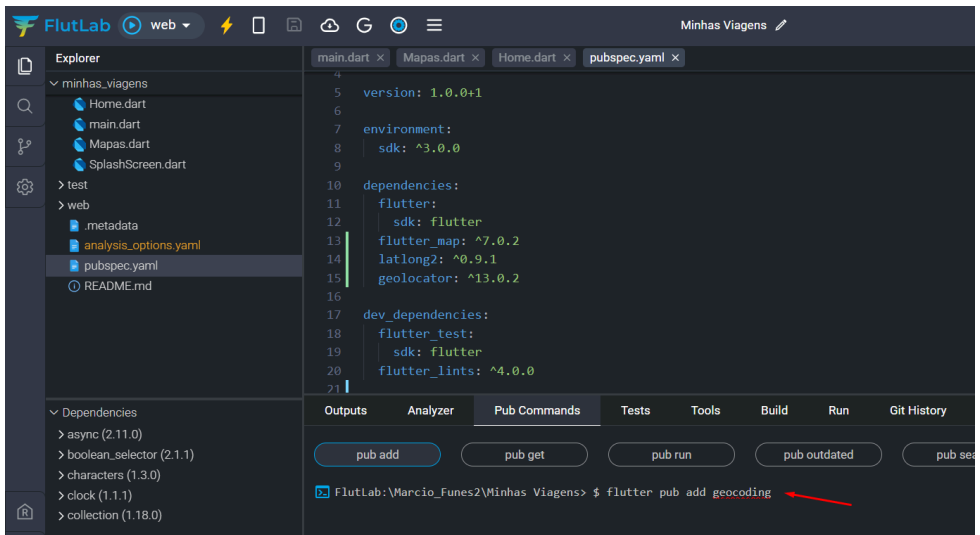
Salvando as Minhas Viagens.....	2
Instalação do pacote geocoding.....	2
Instalação do pacote HTTP.....	4
Adicionando permissões de acesso a Internet.....	5
Model Viagem.....	7
Serviço de Reverse Geocoding.....	9
Home.dart.....	11
main.dart.....	18
Mapas.dart.....	19
Teste Final.....	24
Código-fonte.....	28
geocoding_service.dart.....	28
Home.dart.....	30
main.dart.....	37
Mapas.dart.....	38
SplashScreen.dart.....	44
viagem_model.dart.....	45

Salvando as Minhas Viagens

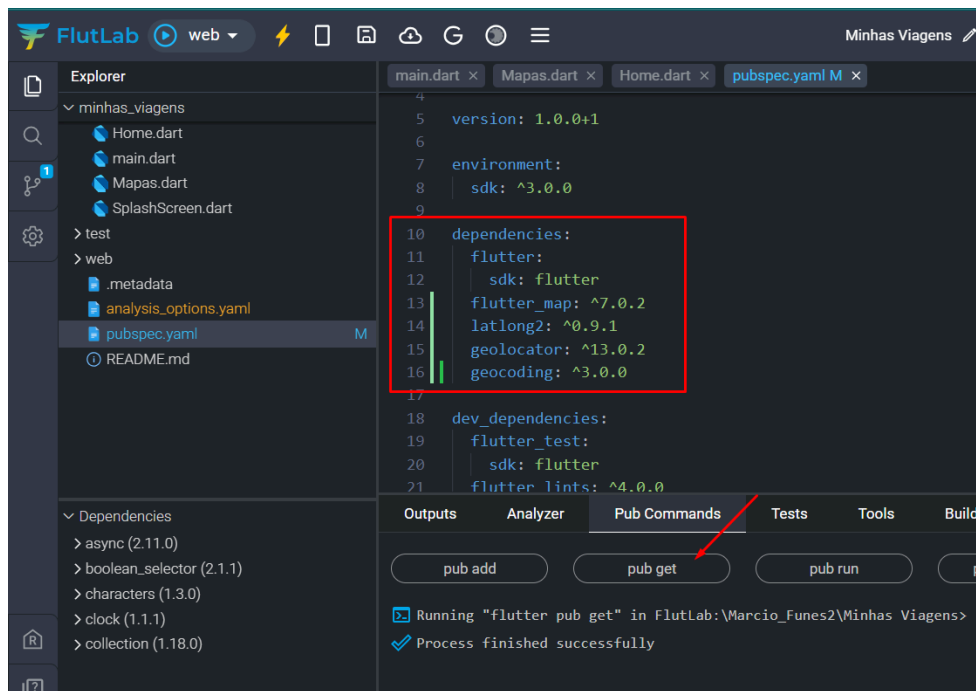
Instalação do pacote geocoding

1 - Após a finalização da parte 1 onde construímos as telas principal, vamos criar a funcionalidade de possibilitar salvar um ponto no mapa em nossa lista pessoal de lugares visitados. Vamos começar instalando uma dependência chamada geocoding:

pub add geocoding

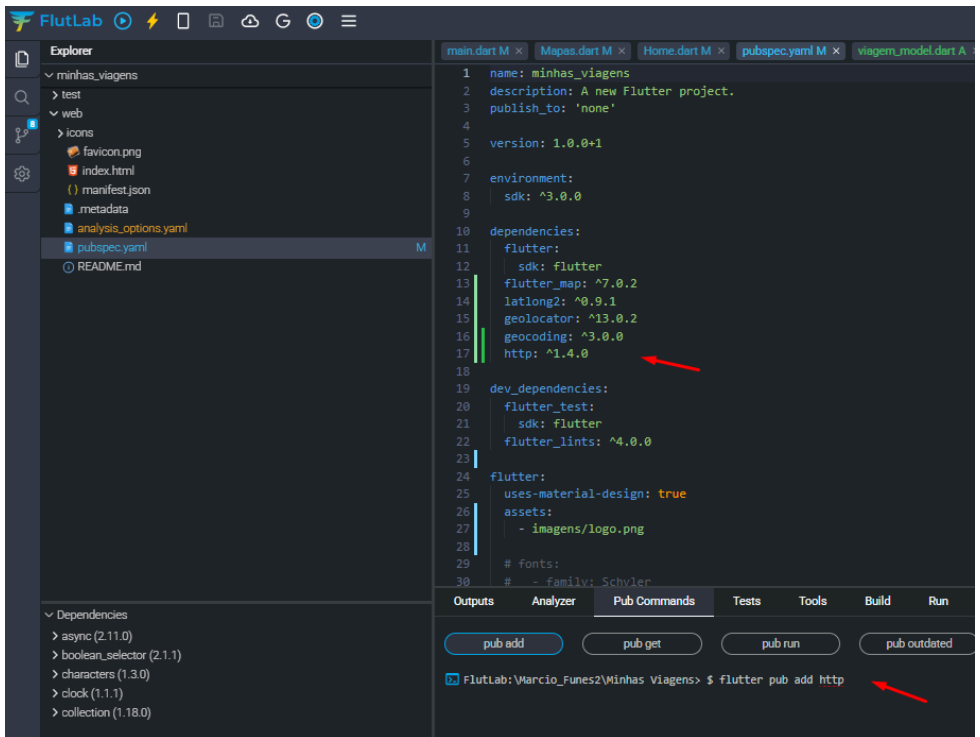


2 - Não esqueça de clicar em pub get para atualizar as dependências no projeto, veja como ficou o pubspec.yaml:



Instalação do pacote HTTP

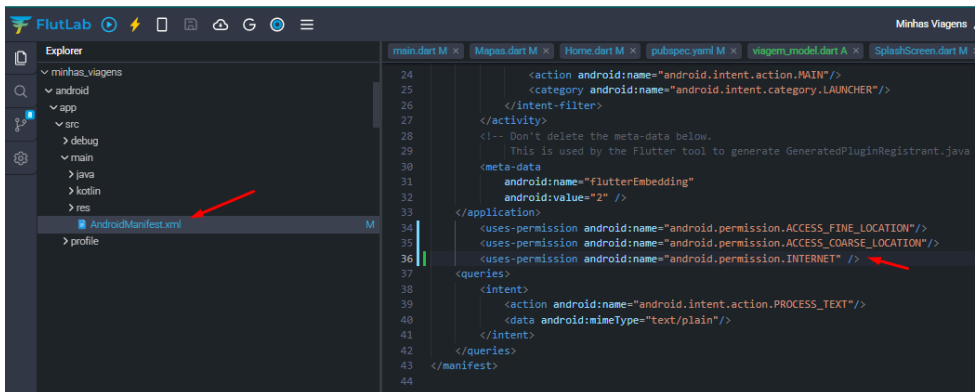
3 - Após selecionarmos as coordenadas ao clicar no mapa vamos usar uma API para obter informações dessas coordenadas, chamamos isso de reverse geocoding. Para essa funcionalidade também faça a instalação do http e não esqueça de clicar em pub get depois da instalação.



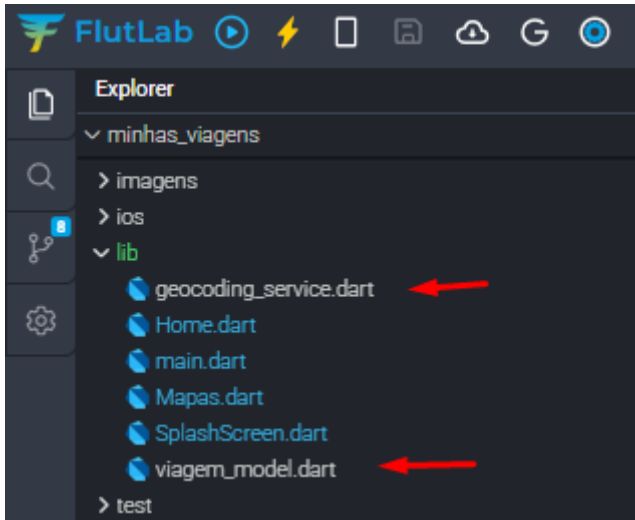
Adicionando permissões de acesso a Internet

4 - Abra o android/app/src/main/AndroidManifest.xml e adicione a permissão de acesso a Internet para usarmos a API de geolocalização.

```
<uses-permission android:name="android.permission.INTERNET" />
```



5 - Para estruturarmos as funcionalidades de geolocalização crie dois arquivos dentro de lib, um chamado geocoding_service.dart e outro viagem_model.dart:



Model Viagem

6 - A classe modelo permite generalizar o que consideramos como uma entidade Viagem em nosso APP especificando quais informações ela contém como coordenadas, nome, endereço e etc:

viagem_model.dart A x

```
1  import 'package:latlong2/latlong.dart';
2
3  class Viagem {
4      final LatLng coordenadas;
5      final String nome;
6      final String endereco;
7      final String cidade;
8      final String estado;
9      final String pais;
10     final String cep;
11     final String? bairro;
12     final String? referencia;
13
14     Viagem({
15         required this.coordenadas,
16         required this.nome,
17         required this.endereco,
18         required this.cidade,
19         required this.estado,
20         required this.pais,
21         required this.cep,
22         this.bairro,
23         this.referencia,
24     });
25
```

```
25
26 factory Viagem.fromJson(Map<String, dynamic> json,
27     {required String nomePersonalizado}) {
28     final endereco = json['address'];
29     return Viagem(
30         coordenadas: LatLng(
31             double.parse(json['lat']),
32             double.parse(json['lon']),
33         ), // LatLng
34         nome: nomePersonalizado,
35         endereco:
36             '${endereco['road'] ?? ''}, ${endereco['house_number'] ?? ''}.trim(),
37         cidade: endereco['city'] ?? endereco['town'] ?? endereco['village'] ?? '',
38         estado: endereco['state'] ?? '',
39         pais: endereco['country'] ?? '',
40         cep: endereco['postcode'] ?? '',
41         bairro: endereco['suburb'] ?? endereco['neighbourhood'] ?? '',
42         referencia: endereco['amenity'] ?? endereco['building'] ?? '',
43     ); // Viagem
44 }
45 }
46
```


Serviço de Reverse Geocoding

7 - Agora vamos criar o serviço de reverse geocoding, abra o arquivo `geocoding_service.dart` e vamos fazer a seguinte implementação. Dessa forma usamos 2 serviços de Mapas, o OpenStreetMaps gratuito que permite visualizar de fato o Mapa na tela e o Google Maps para obter informações de endereço a partir das coordenadas fornecidas pelo OpenStreetMaps:

```
geocoding_service.dart x
1  import 'dart:convert';
2  import 'package:http/http.dart' as http;
3  import 'package:latlong2/latlong.dart';
4
5  class GeocodingService {
6      static const nominatimUrl = 'https://nominatim.openstreetmap.org/reverse';
7
8      static Future<Map<String, dynamic>> getAddressFromCoordinates(
9          LatLng coords) async {
10         final response = await http.get(
11             Uri.parse(
12                 '$nominatimUrl?format=jsonv2&lat=${coords.latitude}&lon=${coords.longitude}'),
13             headers: {'User-Agent': 'YourAppName/1.0'}, // Necessário para Nominatim
14         );
15
16         if (response.statusCode == 200) {
17             return json.decode(response.body);
18         } else {
19             throw Exception('Falha ao obter endereço');
20         }
21     }
22 }
```

```
22
23 static Future<Map<String, dynamic>> getGoogleAddress(
24     LatLng coords, String apiKey) async {
25     final response = await http.get(
26         Uri.parse('https://maps.googleapis.com/maps/api/geocode/json?'
27             'latlng=${coords.latitude},${coords.longitude}&key=$apiKey'),
28     );
29
30     if (response.statusCode == 200) {
31         return json.decode(response.body);
32     } else {
33         throw Exception('Falha ao obter endereço do Google');
34     }
35 }
36 }
37
```

Home.dart

8 - Agora que temos essas funcionalidades prontas vamos atualizar o Home.dart:

```
Home.dart M x
1  import 'package:flutter/material.dart';
2  import 'package:latlong2/latlong.dart';
3  import 'package:geolocator/geolocator.dart';
4  import 'package:geocoding_service.dart';
5  import 'viagem_model.dart';
6  import 'Mapas.dart';
7
8  class Home extends StatefulWidget {
9    @override
10    _HomeState createState() => _HomeState();
11  }
12
13  class _HomeState extends State<Home> {
14    final List<Viagem> _listaViagens = [];
15
16    Future<void> _adicionarLocal() async {
17      final selectedLocation = await Navigator.push<LatLng>(
18        context,
19        MaterialPageRoute(builder: (context) => const Mapas()),
20      );
21
22      if (selectedLocation != null) {
23        await _adicionarViagemDaLocalizacao(selectedLocation);
24      }
25    }
26
27    Future<void> _adicionarViagemDaLocalizacao(LatLng location) async {
28      try {
29        // 1. Obter dados da API
30        final dadosLocal =
31          await GeocodingService.getAddressFromCoordinates(location);
32
33        // 2. Diálogo para nome personalizado
34        final nomeController = TextEditingController(
35          text: dadosLocal['name'] ??
36            dadosLocal['address']['amenity'] ??
37            'Novo Local',
38        );
39      } catch (e) {}
40    }
41  }
```

```

39
40
41 final confirmado = await showDialog<bool>(
42   context: context,
43   builder: (context) => AlertDialog(
44     title: const Text('Confirmar Local'),
45     content: SingleChildScrollView(
46       child: Column(
47         children: [
48           Text(
49             'Endereço: ${dadosLocal['display_name'] ?? 'Não identificado'}', // Text
50             const SizedBox(height: 16),
51           TextFormField(
52             controller: nomeController,
53             decoration: const InputDecoration(
54               labelText: 'Nome para este local',
55             ), // InputDecoration
56           ), // TextFormField
57         ], // Column
58       ), // SingleChildScrollView
59       actions: [
60         TextButton(
61           onPressed: () => Navigator.pop(context, false),
62           child: const Text('Cancelar'),
63         ), // TextButton
64         TextButton(
65           onPressed: () => Navigator.pop(context, true),
66           child: const Text('Salvar'),
67         ), // TextButton
68       ],
69     ), // AlertDialog
70   );

```

```
70     );  
71  
72     if (confirmado == true) {  
73         final novaViagem = Viagem.fromJson(  
74             dadosLocal,  
75             nomePersonalizado: nomeController.text,  
76         );  
77  
78         setState(() => _listaViagens.add(novaViagem));  
79     }  
80 } catch (e) {  
81     // Fallback manual se a API falhar  
82     final nome = await _mostrarDialogoManual(location);  
83     if (nome != null) {  
84         setState(() => _listaViagens.add(  
85             Viagem(  
86                 coordenadas: location,  
87                 nome: nome,  
88                 endereco:  
89                     'Coordenadas: ${location.latitude.toStringAsFixed(6)}, '  
90                     '${location.longitude.toStringAsFixed(6)}',  
91                 cidade: 'Não identificada',  
92                 estado: '',  
93                 pais: '',  
94                 cep: '',  
95             ),  
96         ));  
97     }  
98 }  
99 }  
100 }
```

```

100
101 Future<String?> _mostrarDialogoManual(LatLng location) async {
102     final controller = TextEditingController();
103     return await showDialog<String>(
104         context: context,
105         builder: (context) => AlertDialog(
106             title: const Text('Adicionar Local Manualmente'),
107             content: TextField(
108                 controller: controller,
109                 decoration: const InputDecoration(
110                     labelText: 'Nome do local',
111                     hintText: 'Ex: Meu ponto favorito',
112                 ), // InputDecoration
113             ), // TextField
114             actions: [
115                 TextButton(
116                     onPressed: () => Navigator.pop(context),
117                     child: const Text('Cancelar'),
118                 ), // TextButton
119                 TextButton(
120                     onPressed: () => Navigator.pop(context, controller.text),
121                     child: const Text('Salvar'),
122                 ), // TextButton
123             ],
124         ), // AlertDialog
125     );
126 }
127
128 void _excluirViagem(int index) {
129     setState(() {
130         _listaViagens.removeAt(index);
131     });
132 }
133

```

```

133
134 @override
135 Widget build(BuildContext context) {
136   return Scaffold(
137     appBar: AppBar(
138       title: const Text("Minhas viagens"),
139       actions: [
140         if (_listaViagens.isNotEmpty)
141           IconButton(
142             icon: const Icon(Icons.sort),
143             onPressed: () async {
144               try {
145                 final position = await Geolocator.getCurrentPosition();
146                 setState(() {
147                   _listaViagens.sort((a, b) {
148                     final distanciaA = const Distance().as(
149                       LengthUnit.Kilometer,
150                       LatLng(position.latitude, position.longitude),
151                       a.coordenadas);
152                     final distanciaB = const Distance().as(
153                       LengthUnit.Kilometer,
154                       LatLng(position.latitude, position.longitude),
155                       b.coordenadas);
156                     return distanciaA.compareTo(distanciaB);
157                   });
158                 });
159               } catch (e) {
160                 ScaffoldMessenger.of(context).showSnackBar(
161                   SnackBar(content: Text('Erro ao ordenar: ${e.toString()}')),
162                 );
163               }
164             },
165           ), // IconButton
166       ],
167     ), // AppBar
168     floatingActionButton: FloatingActionButton(
169       child: const Icon(Icons.add),
170       onPressed: _adicionarLocal,

```

```

170         onPressed: _adicionarLocal,
171     ), // FloatingActionButton
172     body: _listaViagens.isEmpty
173     ? const Center(
174         child: Text(
175             'Nenhuma viagem adicionada\nClique no + para começar',
176             textAlign: TextAlign.center,
177             style: TextStyle(fontSize: 18),
178         ), // Text
179     ) // Center
180   : ListView.builder(
181     itemCount: _listaViagens.length,
182     itemBuilder: (context, index) {
183       final viagem = _listaViagens[index];
184       return Card(
185         margin: const EdgeInsets.all(8),
186         child: ListTile(
187           title: Text(
188             viagem.nome,
189             style: const TextStyle(fontWeight: FontWeight.bold),
190           ), // Text
191           subtitle: Column(
192             crossAxisAlignment: CrossAxisAlignment.start,
193             children: [
194               if (viagem.endereco.isNotEmpty) Text(viagem.endereco),
195               if (viagem.cidade.isNotEmpty)
196                 Text('${viagem.cidade}, ${viagem.estado}'),
197               if (viagem.pais.isNotEmpty) Text(viagem.pais),
198               if (viagem.cep.isNotEmpty) Text('CEP: ${viagem.cep}'),
199               Text(
200                 'Coordenadas: ${viagem.coordenadas.latitude.toStringAsFixed(4)}, '
201                 '${viagem.coordenadas.longitude.toStringAsFixed(4)}',
202                 style: const TextStyle(fontSize: 12),
203               ), // Text
204             ],
205           ), // Column
206           trailing: IconButton(
207             icon: const Icon(Icons.delete, color: Colors.red),
208             onPressed: () => _excluirViagem(index),
209           ), // IconButton

```



```
209     ), // IconButton
210     onTap: () {
211       Navigator.push(
212         context,
213         MaterialPageRoute(
214           builder: (context) => Mapas(),
215         ), // MaterialPageRoute
216       );
217     },
218   ), // ListTile
219 ); // Card
220 },
221 ), // ListView.builder
222 ); // Scaffold
223 }
224 }
225
```

main.dart

9 - Atualize o main.dart:

```
main.dart M x
1  import 'package:flutter/material.dart';
2  import 'SplashScreen.dart';
3
4  void main() {
5    runApp(const MyApp());
6  }
7
8  class MyApp extends StatelessWidget {
9    const MyApp({super.key});
10
11    @override
12    Widget build(BuildContext context) {
13      return MaterialApp(
14        title: "Minhas Viagens",
15        theme: ThemeData(
16          primarySwatch: Colors.blue,
17          appBarTheme: const AppBarTheme(
18            backgroundColor: Colors.blue,
19            foregroundColor: Colors.white,
20            centerTitle: true,
21          ), // AppBarTheme
22          floatingActionButtonTheme: const FloatingActionButtonThemeData(
23            backgroundColor: Colors.blue,
24            foregroundColor: Colors.white,
25          ), // FloatingActionButtonThemeData
26        ), // ThemeData
27        home: const SplashScreen(),
28        debugShowCheckedModeBanner: false,
29      ); // MaterialApp
30    }
31  }
32
```

Mapas.dart

10 - Atualize o Mapas.dart:

```
Mapas.dart M x
1  import 'package:flutter/material.dart';
2  import 'package:flutter_map/flutter_map.dart';
3  import 'package:latlong2/latlong.dart';
4  import 'package:geolocator/geolocator.dart';
5
6  class Mapas extends StatefulWidget {
7    const Mapas({Key? key}) : super(key: key);
8
9    @override
10   _MapScreenState createState() => _MapScreenState();
11 }
12
13 class _MapScreenState extends State<Mapas> {
14   final MapController _mapController = MapController();
15   LatLng? _currentLocation;
16   LatLng? _selectedLocation;
17   bool _isLoading = true;
18   String _errorMessage = '';
19   bool _mapInitialized = false;
20
21   @override
22   void initState() {
23     super.initState();
24     _getCurrentLocation();
25   }
26
27   Future<bool> _checkPermissions() async {
28     bool serviceEnabled = await Geolocator.isLocationServiceEnabled();
29     if (!serviceEnabled) {
30       setState(() {
31         _errorMessage = 'Serviço de localização desabilitado';
32         _isLoading = false;
33       });
34       return false;
35     }
36   }
```

```

36
37 LocationPermission permission = await Geolocator.checkPermission();
38 if (permission == LocationPermission.denied) {
39   permission = await Geolocator.requestPermission();
40   if (permission == LocationPermission.denied) {
41     setState(() {
42       _errorMessage = 'Permissão de localização negada';
43       _isLoading = false;
44     });
45     return false;
46   }
47 }
48
49 if (permission == LocationPermission.deniedForever) {
50   setState(() {
51     _errorMessage =
52       'Permissão permanentemente negada. Ative nas configurações.';
53     _isLoading = false;
54   });
55   return false;
56 }
57
58 return true;
59
60
61 Future<void> _getCurrentLocation() async {
62   if (!await _checkPermissions()) return;
63
64   try {
65     Position position = await Geolocator.getCurrentPosition(
66       desiredAccuracy: LocationAccuracy.best,
67     );
68
69     setState(() {
70       _currentLocation = LatLng(position.latitude, position.longitude);
71       _isLoading = false;
72     });
73
74     if (_mapInitialized) {
75       _mapController.move(_currentLocation!, 16);
76     }

```

```

76     }
77   } catch (e) {
78     setState(() {
79       _isLoading = false;
80       _errorMessage = 'Erro ao obter localização: ${e.toString()}';
81     });
82   }
83 }
84
85 void _handleTap(TapPosition tapPosition, LatLng latLng) {
86   setState(() {
87     _selectedLocation = latLng;
88   });
89 }
90
91 @override
92 Widget build(BuildContext context) {
93   return Scaffold(
94     appBar: AppBar(
95       title: const Text("Selecione um Local"),
96       actions: [
97         if (_selectedLocation != null)
98           IconButton(
99             icon: const Icon(Icons.check),
100             onPressed: () {
101               Navigator.pop(context, _selectedLocation);
102             },
103           ), // IconButton
104       ],
105     ), // AppBar
106     body: _buildMapContent(),
107     floatingActionButton: _selectedLocation != null
108       ? FloatingActionButton(
109         child: const Icon(Icons.my_location),
110         onPressed: () {
111           _mapController.move(_selectedLocation!, 16);
112         },
113       ) // FloatingActionButton
114       : null,
115   ); // Scaffold
116 }
117

```

```
117
118 Widget _buildMapContent() {
119   if (_isLoading) {
120     return const Center(child: CircularProgressIndicator());
121   }
122
123   if (_errorMessage.isNotEmpty) {
124     return Center(
125       child: Column(
126         mainAxisAlignment: MainAxisAlignment.center,
127         children: [
128           Text(_errorMessage),
129           const SizedBox(height: 20),
130           ElevatedButton(
131             onPressed: _getCurrentLocation,
132             child: const Text('Tentar Novamente'),
133           ), // ElevatedButton
134           if (_errorMessage.contains('permanentemente'))
135             TextButton(
136               onPressed: () => Geolocator.openAppSettings(),
137               child: const Text('Abrir Configurações'),
138             ), // TextButton
139         ],
140       ), // Column
141     ); // Center
142   }
143 }
```

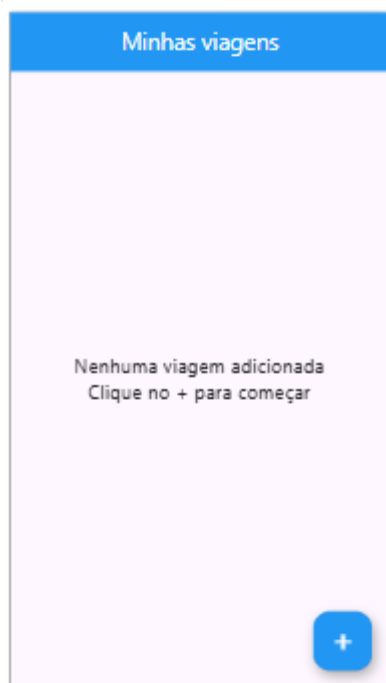
```

143
144 return FlutterMap(
145   mapController: _mapController,
146   options: MapOptions(
147     initialCenter: _currentLocation ?? const LatLng(0, 0),
148     initialZoom: 16,
149     onTap: _handleTap,
150     onMapReady: () {
151       setState(() => _mapInitialized = true);
152       if (_currentLocation != null) {
153         _mapController.move(_currentLocation!, 16);
154       }
155     },
156   ), // MapOptions
157   children: [
158     TileLayer(
159       urlTemplate: 'https://tile.openstreetmap.org/{z}/{x}/{y}.png',
160       userAgentPackageName: 'com.example.app',
161     ), // TileLayer
162     if (_currentLocation != null)
163       MarkerLayer(
164         markers: [
165           Marker(
166             point: _currentLocation!,
167             child: const Icon(Icons.person_pin_circle,
168               size: 50, color: Colors.blue), // Icon
169           ), // Marker
170         ],
171       ), // MarkerLayer
172     if (_selectedLocation != null)
173       MarkerLayer(
174         markers: [
175           Marker(
176             point: _selectedLocation!,
177             child: const Icon(Icons.location_pin, size: 50, color: Colors.red),
178           ), // Marker
179         ],
180       ), // MarkerLayer
181   ],
182 ); // FlutterMap
183 }
184 }
185 }
186

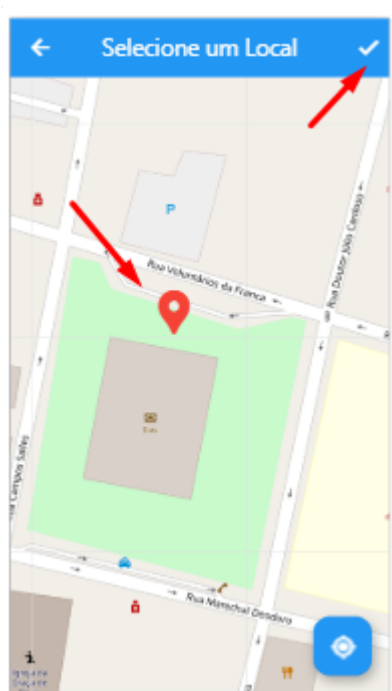
```

Teste Final

11 - Faça o build do App, veja que a lista está vazia. Clique no botão de +



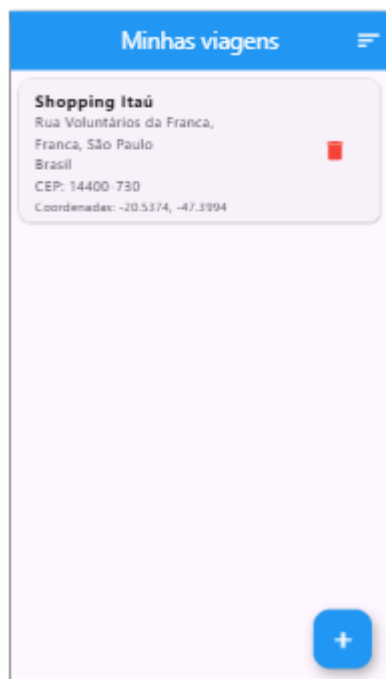
12 - Selecione um ponto desejado no mapa e depois clique no botão de check no canto superior direito:



13 - Veja que nossa chamada de API funcionou e informando as coordenadas obtivemos o endereço, cidade, bairro, cep e etc. Coloque um nome para essa localidade e clique em salvar:



14 - Pronto, veja que nossa lista recebeu essa informação e agora temos o registro dentro do app de locais onde o usuário já visitou. Caso queira, o usuário pode deletar essa entrada ao clicar no ícone vermelho da lixeira.



Código-fonte

geocoding_service.dart

```
import 'dart:convert';
import 'package:http/http.dart' as http;
import 'package:latlong2/latlong.dart';

class GeocodingService {
  static const nominatimUrl =
    'https://nominatim.openstreetmap.org/reverse';

  static Future<Map<String, dynamic>> getAddressFromCoordinates(
    LatLng coords) async {
    final response = await http.get(
      Uri.parse(
        '$nominatimUrl?format=jsonv2&lat=${coords.latitude}&lon=${coords.longitude}',
        headers: {'User-Agent': 'YourAppName/1.0'}, // Necessário para
        Nominatim
      );

    if (response.statusCode == 200) {
      return json.decode(response.body);
    } else {
      throw Exception('Falha ao obter endereço');
    }
  }

  static Future<Map<String, dynamic>> getGoogleAddress(
    LatLng coords, String apiKey) async {
    final response = await http.get(
```

```
Uri.parse('https://maps.googleapis.com/maps/api/geocode/json?'
    'latlng=${coords.latitude},${coords.longitude}&key=$apiKey'),
);

if (response.statusCode == 200) {
    return json.decode(response.body);
} else {
    throw Exception('Falha ao obter endereço do Google');
}
}
}
```

Home.dart

```
import 'package:flutter/material.dart';
import 'package:latlong2/latlong.dart';
import 'package:geolocator/geolocator.dart';
import 'geocoding_service.dart';
import 'viagem_model.dart';
import 'Mapas.dart';
```

```
class Home extends StatefulWidget {
  @override
  _HomeState createState() => _HomeState();
}
```

```
class _HomeState extends State<Home> {
  final List<Viagem> _listaViagens = [];
```

```
Future<void> _adicionarLocal() async {
  final selectedLocation = await Navigator.push<LatLng>(
    context,
    MaterialPageRoute(builder: (context) => const Mapas()),
  );
```

```
  if (selectedLocation != null) {
    await _adicionarViagemDaLocalizacao(selectedLocation);
  }
}
```

```
Future<void> _adicionarViagemDaLocalizacao(LatLng location) async {
  try {
    // 1. Obter dados da API
    final dadosLocal =
      await GeocodingService.getAddressFromCoordinates(location);
```

```

// 2. Diálogo para nome personalizado
final nomeController = TextEditingController(
  text: dadosLocal['name'] ??
    dadosLocal['address']['amenity'] ??
    'Novo Local',
);

final confirmado = await showDialog<bool>(
  context: context,
  builder: (context) => AlertDialog(
    title: const Text('Confirmar Local'),
    content: SingleChildScrollView(
      child: Column(
        children: [
          Text(
            'Endereço: ${dadosLocal['display_name'] ?? 'Não
identificado'}'),
          const SizedBox(height: 16),
          TextFormField(
            controller: nomeController,
            decoration: const InputDecoration(
              labelText: 'Nome para este local',
            ),
          ),
        ],
      ),
    ),
    actions: [
      TextButton(
        onPressed: () => Navigator.pop(context, false),
        child: const Text('Cancelar'),
      ),
      TextButton(
        onPressed: () => Navigator.pop(context, true),
        child: const Text('Salvar'),
      ),
    ],
  ),
);

```

```

        ),
    ],
),
);

if (confirmado == true) {
    final novaViagem = Viagem.fromJson(
        dadosLocal,
        nomePersonalizado: nomeController.text,
    );

    setState(() => _listaViagens.add(novaViagem));
}
} catch (e) {
    // Fallback manual se a API falhar
    final nome = await _mostrarDialogoManual(location);
    if (nome != null) {
        setState(() => _listaViagens.add(
            Viagem(
                coordenadas: location,
                nome: nome,
                endereco:
                    'Coordenadas: ${location.latitude.toStringAsFixed(6)}, '
                    '${location.longitude.toStringAsFixed(6)}',
                cidade: 'Não identificada',
                estado: "",
                pais: "",
                cep: "",
            ),
        ));
    }
}
}
}
}

```

```

Future<String?> _mostrarDialogoManual(LatLng location) async {

```



```

final controller = TextEditingController();
return await showDialog<String>(
  context: context,
  builder: (context) => AlertDialog(
    title: const Text('Adicionar Local Manualmente'),
    content: TextField(
      controller: controller,
      decoration: const InputDecoration(
        labelText: 'Nome do local',
        hintText: 'Ex: Meu ponto favorito',
      ),
    ),
  ),
  actions: [
    TextButton(
      onPressed: () => Navigator.pop(context),
      child: const Text('Cancelar'),
    ),
    TextButton(
      onPressed: () => Navigator.pop(context, controller.text),
      child: const Text('Salvar'),
    ),
  ],
),
);
}

```

```

void _excluirViagem(int index) {
  setState(() {
    _listaViagens.removeAt(index);
  });
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(

```

```

appBar: AppBar(
  title: const Text("Minhas viagens"),
  actions: [
    if (_listaViagens.isNotEmpty)
      IconButton(
        icon: const Icon(Icons.sort),
        onPressed: () async {
          try {
            final position = await Geolocator.getCurrentPosition();
            setState(() {
              _listaViagens.sort((a, b) {
                final distanciaA = const Distance().as(
                  LengthUnit.Kilometer,
                  LatLng(position.latitude, position.longitude),
                  a.coordenadas);
                final distanciaB = const Distance().as(
                  LengthUnit.Kilometer,
                  LatLng(position.latitude, position.longitude),
                  b.coordenadas);
                return distanciaA.compareTo(distanciaB);
              });
            });
          } catch (e) {
            ScaffoldMessenger.of(context).showSnackBar(
              SnackBar(content: Text('Erro ao ordenar: ${e.toString()}')),
            );
          }
        },
      ),
  ],
),
floatingActionButton: FloatingActionButton(
  child: const Icon(Icons.add),
  onPressed: _adicionarLocal,
),

```

```

body: _listaViagens.isEmpty
? const Center(
  child: Text(
    'Nenhuma viagem adicionada\nClique no + para começar',
    textAlign: TextAlign.center,
    style: TextStyle(fontSize: 18),
  ),
)
: ListView.builder(
  itemCount: _listaViagens.length,
  itemBuilder: (context, index) {
    final viagem = _listaViagens[index];
    return Card(
      margin: const EdgeInsets.all(8),
      child: ListTile(
        title: Text(
          viagem.nome,
          style: const TextStyle(fontWeight: FontWeight.bold),
        ),
        subtitle: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            if (viagem.endereco.isNotEmpty) Text(viagem.endereco),
            if (viagem.cidade.isNotEmpty)
              Text('${viagem.cidade}, ${viagem.estado}'),
            if (viagem.pais.isNotEmpty) Text(viagem.pais),
            if (viagem.cep.isNotEmpty) Text('CEP: ${viagem.cep}'),
            Text(
              'Coordenadas:
${viagem.coordenadas.latitude.toStringAsFixed(4)}, '
              '${viagem.coordenadas.longitude.toStringAsFixed(4)}',
              style: const TextStyle(fontSize: 12),
            ),
          ],
        ),
      ),
    ),
  ),
)

```

```
      trailing: IconButton(
        icon: const Icon(Icons.delete, color: Colors.red),
        onPressed: () => _excluirViagem(index),
      ),
    onTap: () {
      Navigator.push(
        context,
        MaterialPageRoute(
          builder: (context) => Mapas(),
        ),
      );
    },
  ),
);
},
),
);
```

main.dart

```
import 'package:flutter/material.dart';
import 'SplashScreen.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: "Minhas Viagens",
      theme: ThemeData(
        primarySwatch: Colors.blue,
        appBarTheme: const AppBarTheme(
          backgroundColor: Colors.blue,
          foregroundColor: Colors.white,
          centerTitle: true,
        ),
        floatingActionButtonTheme: const FloatingActionButtonThemeData(
          backgroundColor: Colors.blue,
          foregroundColor: Colors.white,
        ),
      ),
      home: const SplashScreen(),
      debugShowCheckedModeBanner: false,
    );
  }
}
```

Mapas.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_map/flutter_map.dart';
import 'package:latlong2/latlong.dart';
import 'package:geolocator/geolocator.dart';
```

```
class Mapas extends StatefulWidget {
  const Mapas({Key? key}) : super(key: key);

  @override
  _MapScreenState createState() => _MapScreenState();
}
```

```
class _MapScreenState extends State<Mapas> {
  final MapController _mapController = MapController();
  LatLng? _currentLocation;
  LatLng? _selectedLocation;
  bool _isLoading = true;
  String _errorMessage = "";
  bool _mapInitialized = false;
```

```
@override
void initState() {
  super.initState();
  _getCurrentLocation();
}
```

```
Future<bool> _checkPermissions() async {
  bool serviceEnabled = await Geolocator.isLocationServiceEnabled();
  if (!serviceEnabled) {
    setState(() {
      _errorMessage = 'Serviço de localização desabilitado';
      _isLoading = false;
    });
  }
```

```
    return false;
}
```

```
LocationPermission permission = await Geolocator.checkPermission();
if (permission == LocationPermission.denied) {
    permission = await Geolocator.requestPermission();
    if (permission == LocationPermission.denied) {
        setState(() {
            _errorMessage = 'Permissão de localização negada';
            _isLoading = false;
        });
        return false;
    }
}
```

```
if (permission == LocationPermission.deniedForever) {
    setState(() {
        _errorMessage =
            'Permissão permanentemente negada. Ative nas configurações.';
        _isLoading = false;
    });
    return false;
}
```

```
    return true;
}
```

```
Future<void> _getCurrentLocation() async {
    if (!await _checkPermissions()) return;

    try {
        Position position = await Geolocator.getCurrentPosition(
            desiredAccuracy: LocationAccuracy.best,
        );
    }
}
```

```

setState(() {
  _currentLocation = LatLng(position.latitude, position.longitude);
  _isLoading = false;
});

if (_mapInitialized) {
  _mapController.move(_currentLocation!, 16);
}
} catch (e) {
  setState(() {
    _isLoading = false;
    _errorMessage = 'Erro ao obter localização: ${e.toString()}';
  });
}
}

```

```

void _handleTap(TapPosition tapPosition, LatLng latLng) {
  setState(() {
    _selectedLocation = latLng;
  });
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text("Selecione um Local"),
      actions: [
        if (_selectedLocation != null)
          IconButton(
            icon: const Icon(Icons.check),
            onPressed: () {
              Navigator.pop(context, _selectedLocation);
            },
          ),
      ],
    ),
  );
}

```



```

    ],
  ),
  body: _buildMapContent(),
  floatingActionButton: _selectedLocation != null
    ? FloatingActionButton(
      child: const Icon(Icons.my_location),
      onPressed: () {
        _mapController.move(_selectedLocation!, 16);
      },
    )
    : null,
);
}

```

```

Widget _buildMapContent() {
  if (_isLoading) {
    return const Center(child: CircularProgressIndicator());
  }
}

```

```

if (_errorMessage.isNotEmpty) {
  return Center(
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Text(_errorMessage),
        const SizedBox(height: 20),
        ElevatedButton(
          onPressed: _getCurrentLocation,
          child: const Text('Tentar Novamente'),
        ),
        if (_errorMessage.contains('permanente'))
          TextButton(
            onPressed: () => Geolocator.openAppSettings(),
            child: const Text('Abrir Configurações'),
          ),
      ],
    ),
  );
}

```

```

    ],
  ),
);
}

```

```

return FlutterMap(
  mapController: _mapController,
  options: MapOptions(
    initialCenter: _currentLocation ?? const LatLng(0, 0),
    initialZoom: 16,
    onTap: _handleTap,
    onMapReady: () {
      setState(() => _mapInitialized = true);
      if (_currentLocation != null) {
        _mapController.move(_currentLocation!, 16);
      }
    },
  ),
  children: [
    TileLayer(
      urlTemplate: 'https://tile.openstreetmap.org/{z}/{x}/{y}.png',
      userAgentPackageName: 'com.example.app',
    ),
    if (_currentLocation != null)
      MarkerLayer(
        markers: [
          Marker(
            point: _currentLocation!,
            child: const Icon(Icons.person_pin_circle,
              size: 50, color: Colors.blue),
          ),
        ],
      ),
    if (_selectedLocation != null)
      MarkerLayer(

```

```
markers: [
  Marker(
    point: _selectedLocation!,
    child:
      const Icon(Icons.location_pin, size: 50, color: Colors.red),
  ),
],
);
}
```

SplashScreen.dart

```
import 'dart:async';
import 'package:flutter/material.dart';
import 'Home.dart';

class SplashScreen extends StatefulWidget {
  const SplashScreen({Key? key}) : super(key: key);

  @override
  State<SplashScreen> createState() => _SplashScreenState();
}

class _SplashScreenState extends State<SplashScreen> {
  @override
  void initState() {
    super.initState();
    Future.delayed(const Duration(seconds: 5), () {
      if (mounted) {
        Navigator.pushReplacement(
          context,
          MaterialPageRoute(builder: (_) => Home()),
        );
      }
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Container(
        color: Color(0xff0066cc),
        padding: EdgeInsets.all(60),
        child: Center(
          child: Image.asset("imagens/logo.png"),
        ),
      ),
    );
  }
}
```

viagem_model.dart

```
import 'package:latlong2/latlong.dart';

class Viagem {
  final LatLng coordenadas;
  final String nome;
  final String endereco;
  final String cidade;
  final String estado;
  final String pais;
  final String cep;
  final String? bairro;
  final String? referencia;

  Viagem({
    required this.coordenadas,
    required this.nome,
    required this.endereco,
    required this.cidade,
    required this.estado,
    required this.pais,
    required this.cep,
    this.bairro,
    this.referencia,
  });

  factory Viagem.fromJson(Map<String, dynamic> json,
    {required String nomePersonalizado}) {
    final endereco = json['address'];
    return Viagem(
      coordenadas: LatLng(
        double.parse(json['lat']),
        double.parse(json['lon']),
```

```
),
nome: nomePersonalizado,
endereço:
    '${endereço['road'] ?? ''}, ${endereço['house_number'] ?? ''}'.trim(),
cidade: endereço['city'] ?? endereço['town'] ?? endereço['village'] ?? '',
estado: endereço['state'] ?? '',
país: endereço['country'] ?? '',
cep: endereço['postcode'] ?? '',
bairro: endereço['suburb'] ?? endereço['neighbourhood'] ?? '',
referência: endereço['amenity'] ?? endereço['building'] ?? '',
);
}
}
```